

任务一：测量前缀缓存

实验步骤：

Step1:

根据题目要求，构造两组负载，每组 32 条测量请求，输入长度均为 2112 个 token

参数	共享前缀组	分散前缀组
单条输入长度	2112	2112
组成	2048 共享前缀+64 独立后缀	2112(首 token 两两不同)
是否预热	是	否
请求条数	32	32
输出长度	16	16
并发上限	8	8

Step2:

利用脚本(bench.py)完成，流程如下：

- 1) 服务预热（5 条短请求，不计入结果）
- 2) 通过 POST /flush_cache 清缓存
- 3) 进行前缀预热，发 1 条只含 2048 共享的 token，打印服务端回报的 cached_tokens
- 4) 发送 32 条测量请求，通过 POST /generate 接口发送 input_ids(数据来自 loads.json)
- 5) 生成结果

Step3:

记录实验数据：

逐一请求结果记录到 requests.csv

汇总结果记录到 summary.csv

主表记录两组负载成功率、吞吐量、缓存命中率、实际执行 Prefill 的 token 数，以及 TTFT、TPOT 和端到端延迟(E2E)的 p50、p95，记录到 run-N-summary_all.csv，以第一次(run-1)结果为参考：

指标	共享前缀组	分散前缀组
成功率	100%	100%
吞吐量(requests/s)	59.549	9.647
缓存命中率	96.97%	0.00%
实际 prefill token 数	2048	67584
TTFT p50(ms)	38.613	436.722
TTFT p95(ms)	45.241	690.077
TPOT p50(ms)	6.623	25.914
TPOT p95(ms)	6.894	43.657
E2E p50(ms)	135.175	826.908
E2E p95(ms)	142.239	894.815

两组测量请求均全部成功，且输入、输出长度一致；共享前缀组具有更高的缓存命中率和更少的实际 Prefill token

分析与问答题目问题:

一. 公式分析:

因果自注意力公式:

$$Attention(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

对于序列中的位置 i 的 token，对应的 K_i 和 V_i 是由 token 的隐藏层状态 h_i 经线性变换计算得到的:

$$K_i = h_i W_K$$

$$V_i = h_i W_V$$

二. 问题回答:

Q1:相同 token 与位置的前缀为什么可以复用 K/V?

A1:

首先，因果掩码使得位置 i 只能注意到 i 前面的位置，使模型看不到未来的信息，因此 h_i 只由前 i 个 token 决定，与他后面的 token 无关，所以一旦 h_i 算出， K_i 和 V_i 就是定值，不会因为后续的内容不同而改变， K_i 和 V_i 就会被缓存下来。

这样，相同 token 与位置的元素，不用再重新计算 K/V 矩阵，而是直接从缓存中使用。

Q2:命中后仍需计算哪些 token?

A2:

无需计算的部分：缓存命中区域内的 token

需要计算的部分：

- 1) 未命中的后缀 token，即新请求中从缓存中断开的新增 token，需要进行 Prefill 计算，生成对应的 Q/K/V
- 2) 自回归生成的 token，模型在 Prefill 阶段结束后，开始逐个预测并生成的未来 token，仅计算当前这个新 token 的 Q/K/V,并加入缓存中

Q3:TTFT 与 TPOT 为什么呈现不同变化?

A3:

TTFT(大幅下降): TTFT 衡量的是从发送请求到收到第一个输出 token 的时间，其核心是 Prefill 阶段。该阶段原本需要并行处理整条 Prompt，是计算密集型的。启用前缀缓存后，大量重复的前缀计算被消除，实际 Prefill 的 token 数量锐减。这相当于砍掉了最繁重的前置计算，因此 TTFT 会呈现断崖式的下降。

TPOP(几乎不变): 衡量的是自回归生成阶段的平均速度，即 Decode 阶段。Decode 阶段每次只计算 1 个 token，是内存带宽受限型的。前缀缓存命中后，生成的后缀文本依然需要逐 token 进行 Attention 计算，计算量级并没有发生本质改变。唯一的微小变化是由于缓存命中导致 Prompt 变短，生成的后缀 token 数量可能微乎其微地影响后续每步的 Attention 矩阵长度，因此 TPOT 基本保持稳定。

三. 数据差异分析

缓存命中率分析：未命中的正是每条请求各自独自的 64 后缀 token，共享的 2048 个 token 全部命中

实际 Prefill token 数分析：对于共享前缀组，cached_tokens 的值非常大。在相同的总请求负载下，共享前缀组需要重新计算的差异化部分极少，因此实际执行的 Prefill token 数量远低于分散前缀组

TTFT 分析：

TTFT = 服务端排队等待+Prefill 计算+第一个 token 的 decode+ 框架开销

共享前缀组：不存在排队争抢。每条请求仅需 Prefill 64 个 token，8 条并发合计也只有 512 个 token，对 GPU 而言可以忽略，批次内因而不存在争抢

分散前缀组：批次内依次排队。每条需 Prefill 2112 个 token，8 条同时到达时服务端的 Prefill 算力被依次瓜分，第 k 条请求必须等待

TPOT 分析：Prefill 分块与 decode 交织，同一批的 8 条请求几乎同时结束，但先完成 Prefill 的请求在 decode 阶段被其他请求未完成的 Prefill 分走了算力，其单 token 耗时因此被拉长。反之，共享前缀组每条请求只有 64 个 token 需要 Prefill，批次内几乎不存在长 Prefill 的争抢，TPOT 因而稳定在较低水平。

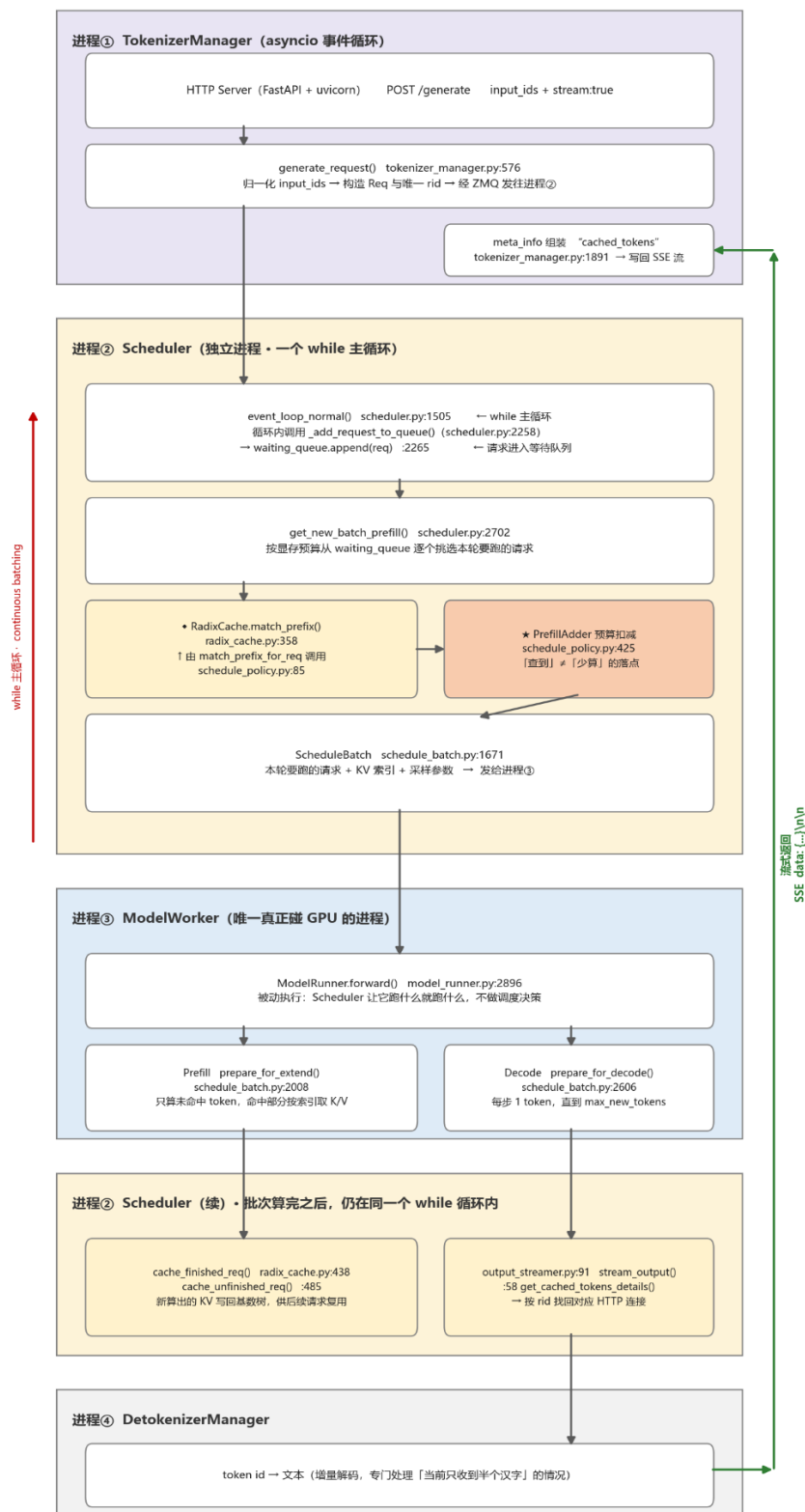
E2E 分析：由于 $E2E = TTFT + TPOT * (N-1)$ 受 TTFT 影响，共享前缀组会比分散前缀组更低

（任务二在下一页）

任务二：阅读 SGLang 请求流程

SGLang 0.5.14: 一次 /generate 请求的完整流程

四个进程 · ZMQ 消息传递 · 每个关键节点标出所在文件与行号



★ = 前缀命中转化为「少算」的关键落点 * = 关键匹配节点 浅黄 = 与 KV 缓存直接相关的节点 所有路径与行号均经实测 grep 验证

一. 说明:

进程 1

HTTP 层收到 POST /generate 后调用 tokenizer_manager.py 的 generate_request()函数, 该函数把请求体中的 input_ids 归一化为 token 序列, 构造 Req 对象分配唯一请求号 rid, 随后发往 ZMQ 发往 Scheduler 进程。Scheduler 是一个独立进程, scheduler.py 中其主循环 event_loop_normal()每一轮首先先收取新请求, 调用_add_request_to_queue()将其追加进等待队列:

```
self.waiting_queue.append(req)
```

进程 2

每轮循环中, scheduler.py 中的 get_new_batch_prefill()从 waiting_queue 逐个挑选请求组成本轮批次。挑选时会对每个候选请求调用 schedule_policy.py 的 match_prefix_for_req(), 后者进而调用 radix_cache.py 中的 RadixCache.match_prefix()。该函数在一颗压缩前缀树中自根向下查找最长的已缓存前缀, 返回命中的 token 数量以及这些 KV 在显存中的位置, 返回值最终成为响应中的 cached_tokens 字段, 接着通过 schedule_policy.py 的 prefillAdder 的预算逻辑: 它逐个把请求加入批次并累加成本, 扣减未命中的 token 数, 一旦超出 max_prefill_tokens 便停止接纳, 其余请求等待下一轮。

进程 3

批次就绪后交给 ModelWorker, 由 model_runner.py 的 ModelRunner.forward()执行: ①Prefill 阶段: schedule_batch.py 中 prepare_for_extend()未命中的 token 计算 K/V, 命中部分的 KV 按索引直接从缓存取用, 不再重算。②Decade 阶段: dchedule_batch.py 中 prepare_for_decode()逐步生成输出 token。新计算的 KV 通过 RadixCache 的两个入口写回基数树: radix_cache.py 中的 cache_unfinished_req()和 cache_finished_req(), 前者在请求仍在进行时, 把已算完的 KV 先挂进树, 使其它请求提前复用, 后者在请求结束后, 把完整的 KV 正式插入基数数, 供后续请求复用。当显存不足时, 树上的节点按淘汰策略被驱逐, 为新请求腾出空间。

进程 4

输出环节由 out_streamer.py 的 stream_output()负责, 已生成的 token 先还原成文本, 一个汉字可能被切成多个 token, 流式输出时必须处理只收到半个字符的情形, 否则会输出乱码, 文本随后按 rid 找回对应的 HTTP 连接, 由 TokenManager 组装 tokenizer_manager.py 的 meta_info 后以 data:{...}的格式写出, HTTP 层异步生成器逐块推送至客户端

二. 函数文件位置:

用到的函数均在 `sglang/srt/...` 中

函数	位置
<code>generate_request</code>	<code>managers/tokenizer_manager.py</code>
<code>event_loop_normal</code>	<code>managers/scheduler.py</code>
<code>event_loop_overlap</code>	<code>managers/scheduler.py</code>
<code>get_new_batch_prefill</code>	<code>managers/scheduler.py</code>
<code>match_prefix_for_req</code>	<code>managers/schedule_policy.py</code>
<code>RadixCache.match_prefix</code>	<code>mem_cache/radix_cache.py</code>
<code>PrefillAdder</code>	<code>managers/schedule_policy.py</code>
<code>ScheduleBatch</code>	<code>managers/schedule_batch.py</code>
<code>cache_finished_req</code>	<code>mem_cache/radix_cache.py</code>
<code>cache_unfinished_req</code>	<code>mem_cache/radix_cache.py</code>
<code>stream_output</code>	<code>managers/scheduler_components/output_streamer.py</code>

三. 问题回答:

Q1:请求如何进入等待队列?

A1:

HTTP 层收到 POST `/generate` 后调用 `generate_request()`, 把 `input_ids` 归一化为 token 序列, 构造 Req 对象并分配唯一 rid, 经 ZMQ 发往 Scheduler 进程。Scheduler 是独立进程, 主循环 `event_loop_normal()` 每一轮先收新请求, 由 `_add_request_to_queue()` 执行 `self.waiting_queue.append(req)` 请求随后停留在等待队列。

Q2:前缀匹配如何减少本轮 Prefill token?

A2:

`get_new_batch_prefill()` 从等待队列逐个挑选请求组批, 挑选时对每个候选调用 `match_prefix_for_req()` → `RadixCache.match_prefix()`。该函数在一棵压缩前缀树中自根向下查找最长的已缓存前缀, 返回命中 token 数量与对应 KV 在显存中的位置, 这个值最终成为响应里的 `cached_tokens`。PrefillAdder 把命中转化为计算量减少: 它逐个把请求加入批次并累加成本, 扣减的是未命中部分的 token 数, 一旦超出显存上限或 `max_prefill_tokens` 便停止接纳, 其余请求留待下一轮。

Q3:新产生的 KV 如何写回缓存

A3:

批次就绪后交给 `ModelRunner.forward()` 执行: Prefill 阶段只为未命中的 token 计算 K/V, 命中部分的 KV 按索引直接从缓存取用, 不再重算。Decade 阶

段逐步生成输出 token。新计算的 KV 通过 RadixCache 的两个入口写回基数树：cache_unfinished_req()和 cache_finished_req()，前者在请求仍在进行时，把已算完的 KV 先挂进树，使其它请求提前复用，后者在请求结束后，把完整的 KV 正式插入基数数，供后续请求复用。当显存不足时，树上的节点按淘汰策略被驱逐，为新请求腾出空间。

Q4:生成结果如何流式返回？

A4:

输出环节由 out_streamer.py 的 stream_output()负责，已生成的 token 先还原成文本，一个汉字可能被切成多个 token，流式输出时必须处理只收到半个字符的情形，否则会输出乱码，文本随后按 rid 找回对应的 HTTP 连接，由 TokenManager 组装 tokenizer_manager.py 的 meta_info 后以 data:{...}的格式写出，HTTP 层异步生成器逐块推送至客户端